



Git Branch

Introduction

Imagine you are working on a **team project**. You are assigned to develop a **new feature**, but you don't want to affect the existing code until the feature is fully ready.

How can you work on new features without modifying the main project?

This is where the **git branch** command helps!

Section 1: Learn

What is **git branch**?

git branch is a Git command used to create, list, and manage branches in a repository. A branch allows developers to:

1. **Work on new features without modifying the main project.**
2. **Create separate versions** of the project to test changes.
3. **Merge changes into the main project when ready.**

Why Do We Need **git branch**?

Without Branches	With Branches (git branch)
All developers work on the same code	Each developer works in their own branch
Risk of breaking the main code	Changes are isolated, preventing errors
Hard to test new features	Test features in a separate branch



Without Branches	With Branches (git branch)
Difficult to collaborate	Multiple developers can work simultaneously

How Does **git branch** Work?

1. You create a new branch using **git branch new-feature**.
2. Switch to the branch using **git checkout new-feature**.
3. Make changes and commit them to the branch.
4. Merge the branch into the main project when done.

An Interesting Anecdote

Before Git, developers created multiple copies of the project folder to work on new features. This caused confusion, duplication, and lost changes. With Git branching, teams can work in parallel without affecting the main project.

Section 2: Practice

Step-by-Step Guide to Using **git branch**

1. Installing Git (If Not Installed)

```
# Check if Git is installed
```

```
git --version
```

```
# If Git is not installed, install it (Ubuntu)
```

```
sudo apt update
```

```
sudo apt install git
```



Why is this important?

- Ensures Git is installed and ready to use.
 - Allows you to create and manage branches.
-

2. Checking Available Branches

```
# List all branches in the repository
```

```
git branch
```

What happens here?

- Displays all branches in the repository.
 - The active branch is marked with *****.
-

3. Creating a New Branch

```
# Create a new branch called "feature-1"
```

```
git branch feature-1
```

Why is this useful?

- Allows you to work on a new feature separately.
 - The main code remains unchanged until the feature is complete.
-

4. Switching to a Branch

```
# Switch to the new branch
```

```
git checkout feature-1
```

```
# OR (Newer way to switch branches)
```



```
git switch feature-1
```

What does this do?

- Moves you to the feature-1 branch, so any changes are made there.
 - Prevents accidental changes to the main branch.
-

5. Creating and Switching to a Branch in One Command

```
git checkout -b feature-1
```

```
# OR (Newer way)
```

```
git switch -c feature-1
```

Why is this useful?

- Saves time by creating and switching to a branch in one step.
-

6. Merging a Branch into the Main Project

After working on a feature, you merge it into the main branch.

```
# Switch to the main branch
```

```
git checkout main
```

```
# Merge the feature branch
```

```
git merge feature-1
```

What does this do?

- Combines the changes from feature-1 into the main branch.
 - The feature is now part of the main project.
-



7. Deleting a Branch After Merging

```
# Delete a local branch
```

```
git branch -d feature-1
```

Why is this important?

- Keeps the repository clean by removing old branches.
-

Real-World Example

A software company is developing a mobile banking app. The team has separate tasks:

- Developer A works on the login system.
- Developer B develops the fund transfer feature.
- Developer C fixes security vulnerabilities.

Without Branching:

- All developers work in the same codebase, causing conflicts and errors.
- A small mistake can break the entire app.

With Branching (**git branch**):

1. Each developer creates a separate branch for their task.
2. Changes are made without affecting the main project.
3. Once tested, each branch is merged into the main branch.

Result? Faster development, fewer conflicts, and smooth teamwork.



Section 3: Know More

Frequently Asked Questions

1. How do I rename a branch?

```
git branch -m old-branch-name new-branch-name
```

2. Can I delete a branch that is not merged?

```
git branch -D feature-1
```

3. The **-D** flag forces deletion even if not merged.

4. How do I switch back to the previous branch?

```
git checkout -
```

5. This quickly switches back to the last branch you were on.
6. What is the difference between **git branch** and **git checkout**?
 - **git branch** creates a branch, but does not switch to it.
 - **git checkout** switches to an existing branch.
7. Where can I practice Git branching?
 - Create a GitHub repository and test branching.
 - Use GitHub Desktop for a graphical interface.

Conclusion

The **git branch** command is essential for managing multiple features and working in teams. It allows developers to create, switch, and merge branches, improving workflow and avoiding errors.

Start by creating and switching branches, experiment with merging, and soon you'll be collaborating on real-world projects with ease!